UNIVERSITAS PAHLAWAN TUANKU TAMBUSAI
FAKULTAS TEKNIK — PROGRAM STUDI TEKNIK INFORMATIKA

# LEMBAR KERJA MAHASISWA

Pertemuan 6 · Pewarisan dan Method Khusus

| | | | |
|---|---|---|---|
| **Nama** | MUHAMMAD SYAIKHUL SYAFIQ | **NIM** | 2555201009 |
| **Kelas** | PBO KELAS A | **Tanggal** | 05-11-2026 |
| **Tautan repositori** | | | |

## A. KUIS AWAL — MENJEMBATANI DARI PERTEMUAN 5 (10 MENIT)

1. **Sebutkan uji pembeda antara komposisi dan agregasi.**

Uji pembedanya terletak pada siklus hidup objek bagian. Pada komposisi, objek bagian tidak bisa berdiri sendiri dan akan ikut hancur jika objek utama dihapus (hubungan mutlak). Sebaliknya, pada agregasi, objek bagian dapat tetap hidup dan berdiri sendiri meskipun objek utamanya sudah dihapus atau dibubarkan.

2. **Mengapa BarisSewa menyimpan salinan tarif, padahal ia sudah menyimpan rujukan ke alatnya?**

Penyimpanan salinan tarif bertujuan untuk menjaga konsistensi data historis transaksi. Jika harga atau tarif alat mengalami perubahan di masa depan, catatan transaksi yang sudah berlalu tidak akan ikut berubah, sehingga laporan keuangan atau riwayat penyewaan tetap akurat.

3. **Mengapa property baris mengembalikan tuple, bukan daftar?**

Tujuannya untuk menerapkan prinsip enkapsulasi dan imutabilitas, agar data transaksi tidak bisa diubah secara sembarangan dari luar kelas. Dengan menggunakan tuple, struktur data menjadi lebih aman dari modifikasi yang tidak sah.

**4. Tulis multiplisitas relasi Sewa – BarisSewa di kedua ujungnya.**

- Sewa → BarisSewa: 1..*
- BarisSewa → Sewa: 1
- *Artinya:* Satu transaksi Sewa mencakup satu atau lebih objek BarisSewa, sedangkan satu objek BarisSewa secara spesifik hanya terikat pada satu transaksi Sewa.

## B. PENGAMATAN PRAKTIKUM

### B.1 super() yang terlupa (Langkah 3)

| Pertanyaan | Jawaban Anda |
|---|---|
| Apakah objeknya berhasil dibuat? Apakah ada galat saat itu? | Objek berhasil diinisialisasi tanpa memunculkan galat saat pertama kali dibuat. Namun, atribut dari kelas induk gagal diinisialisasi karena pemanggilan super().__init__() terlewat. |
| Apa isi vars(t), dan berapa atribut yang seharusnya ada? | Isi vars(t) hanya menampilkan atribut milik kelas anak saja (misalnya {'tenaga_kuda': 8}). Seharusnya atribut kelas induk (seperti kode, nama, dan tarif) juga ikut tercakup, sehingga jumlah atribut totalnya lebih banyak dari yang tercetak saat ini. |
| Method mana yang runtuh, dan atribut apa yang dicarinya? | - Method keterangan() runtuh karena gagal menemukan atribut kode dan nama.<br>- Method biaya_sewa() runtuh karena gagal menemukan atribut tarif. |
| Dengan kalimat sendiri: apa sesungguhnya yang dikerjakan super().__init__()? | `super().__init__() berfungsi untuk meneruskan parameter dan menjalankan konstruktor yang ada di kelas induk (superklass), sehingga kelas anak mendapatkanwarisan atribut dan inisialisasi yang sah dari induknya.` |

### B.2 __eq__tanpa __hash__(Langkah 7)

| Pertanyaan | Jawaban Anda |
|---|---|

| | |
|---|---|
| Berapa nilai AlatTani.__hash__yang ditampilkan? | Nilainya adalah None, karena Python secara otomatis menonaktifkan fungsi hash jika kelas mendeklarasikan __eq__ secara kustom tanpa mendefinisikan __hash__ secara eksplisit. |
| Tiga tindakan apa yang gagal, dan apa pesan galatnya? | ⍰ *Memasukkan objek ke dalam set:* Gagal dengan pesan cannot use 'Alat' as a set element (unhashable type: 'Alat').<br>⍰ *Menjadikan objek sebagai kunci dict:* Gagal dengan pesan cannot use 'Alat' as a dict key (unhashable type: 'Alat').<br>⍰ *Mencari objek menggunakan in pada set:* Gagal dengan pesan cannot use 'Alat' as a set element (unhashable type: 'Alat'). |
| Mengapa Python mematikan __hash_ begitu __eq__ditulis ulang? | Karena dalam aturan struktur data berbasis hash (seperti set atau dictionary), objek yang bernilai sama (==) wajib memiliki nilai hash yang identik. Mengubah __eq__ tanpa menyesuaikan aturan hash berisiko merusak konsistensi pencarian data, sehingga Python memilih menonaktifkannya secara otomatis agar aman. |
| Aturan apa yang harus dipenuhi __eq_ dan __hash__agar konsisten? | Prinsip utamanya: jika dua objek dinilai setara berdasarkan __eq__ (a == b), maka nilai hash keduanya wajib sama (hash(a) == hash(b)). Sebaliknya, dua objek yang memiliki hash sama belum tentu harus bernilai sama. |

## B.3 Dua cara perkakas menangkap kesalahan

| Berkas | Kode Aturan Ruff | Menunjuk Gejala atau Penyakit? |
|---|---|---|
| `lupa_super.py` | ARG002 (3 parameter tidak digunakan) | Menunjuk gejala, yaitu parameter kode, nama, dan tarif tidak digunakan. Penyebabnya adalah lupa memanggil super().__init__(). |
| `eq_tanpa_hash.py` | PLW1641 | Menunjuk masalah secara langsung, yaitu kelas memiliki __eq__() tetapi tidak memiliki __hash__(). |

1. **Ruff menangkap keduanya, tetapi dengan cara yang berbeda. Jelaskan perbedaannya, dan apa artinya bagi cara Anda membaca peringatan perkakas.**
    - **Perbedaan Analisis:**
        - **Pada berkas lupa_super.py, Ruff tidak mendeteksi kesalahan logika pewarisan secara langsung, melainkan mendeteksi gejala sekunder berupa variabel atau parameter yang didefinisikan tetapi tidak pernah dipakai (*unused arguments*, kode galat ARG002). Hal ini terjadi karena parameter kode, nama, dan tarif dikirim ke kelas anak, namun karena super().__init__() tidak dipanggil, parameter tersebut mangkrak dan tidak diolah oleh konstruktor induk.**
        - **Sebaliknya, pada berkas eq_tanpa_hash.py, Ruff mendeteksi akar masalah utama secara langsung menggunakan aturan statis khusus (PLW1641) yang mengenali bahwa sebuah kelas mengimplementasikan method __eq__() tanpa mendampinginya dengan __hash__().**
    - **Implikasi/Artinya bagi pengembang: Peringatan dari perkakas *linter* (seperti Ruff) harus dibaca dengan pemahaman konteks logika pemrograman, bukan sekadar diikuti secara mentah. Terkadang, galat pemrograman tingkat tinggi (seperti lupa memanggil inisialisasi superclass) hanya tampak di permukaan sebagai peringatan kode gaya atau variabel tak terpakai (*gejala*). Oleh karena itu, kita harus menelusuri**

**alur kerja kode dari akarnya, bukan hanya memperbaiki gejala yang disuarakan oleh linter.**

.

## C. AKTIVITAS KELOMPOK: KAPAN PEWARISAN MERUGIKAN

Anggota kelompok: _____

AHMAD RIDHO

RAFIQ HILMI

REYHAN PRATAMA

MUHAMMAD SYAIKHUL SYAFIQ

### C.1 Uji "adalah sejenis" pada proyek kelompok

| Pasangan Kelas | Kalimat "A adalah sejenis B" | Masuk Akal? Relasi yang Tepat? |
|---|---|---|
| Traktor – AlatTani | Traktor adalah sejenis AlatTani | Ya |
| PompaAir – AlatTani | PompaAir adalah sejenis AlatTani | Ya |
| Perontok – AlatTani | Perontok adalah sejenis AlatTani | Ya |

### C.2 Kemiripan yang sebaiknya diselesaikan dengan komposisi

Sebutkan satu, beserta alasannya. Butir ini bernilai paling tinggi.

Hubungan antara Sewa dan BarisSewa sebaiknya menggunakan komposisi, bukan pewarisan. Alasannya, BarisSewa merupakan bagian dari Sewa yang berisi rincian alat dan biaya penyewaan. BarisSewa bukan jenis dari Sewa, melainkan bagian yang dimiliki oleh Sewa.

## D. REFLEKSI PENUTUP

1. **Validasi tarif yang Anda tulis di Pertemuan 3 kini menjaga tiga kelas baru tanpa satu baris pun disalin. Menurut Anda apa risikonya bila suatu hari validasi itu perlu berbeda untuk salah satu turunan?**

**Risiko utamanya adalah pelanggaran prinsip pewarisan (jika menggunakan *override* yang tidak konsisten) atau terjadinya duplikasi kode terpaksa jika kelas turunan tersebut harus mengabaikan validasi induknya. Solusinya adalah mendesain ulang method validasi di kelas induk agar mendukung polimorfisme (misalnya menggunakan *hook method* atau parameter fleksibel) sehingga aturan khusus turunan dapat diakomodasi tanpa merusak struktur kelas utama.**

2. **Sebutkan satu hal yang belum Anda pahami, dan pertanyaan untuk Pertemuan 7.**
   - *Hal yang belum dipahami:* Batasan ideal dalam merancang hierarki kelas agar tidak terjadi pewarisan bertingkat yang terlalu dalam (*deep inheritance tree*).
   - *Pertanyaan untuk Pertemuan 7:* Bagaimana cara menerapkan konsep polimorfisme tingkat lanjut pada antarmuka kelas yang kompleks tanpa membuat ketergantungan antar modul menjadi tinggi?

| Paraf Asisten / Dosen | Tanda Tangan Mahasiswa |
|---|---|
| …… | …… |